



**UNIVERSIDAD
DE LA FRONTERA**

Facultad de Ingeniería y Ciencias

*****COMPUTACIÓN Y PROGRAMACIÓN CUÁNTICA*****

**Evaluación experimental del algoritmo de Grover en
búsqueda exhaustiva: comparación con métodos clásicos.
“Q-Search - Informe final”**

Integrantes:

Francisco Antonio Barrozo Contreras
Antonia Paz Paredes León

Docente:

Dr. Samuel Eduardo Sepúlveda Cuevas

Temuco, 06 de julio de 2026.

Índice

Índice	2
Resumen	3
Introducción	4
Análisis del caso de estudio y su relevancia	5
Descripción del caso	5
Delimitación metodológica: tres niveles de análisis	6
Nivel 1 - Simulación ideal (qiskit Aer, $n \leq 30$)	6
Nivel 2 - Simulación con análisis de recursos (Qiskit Aer + QRydDemo, $n = 2$ a 6)	6
Nivel 3 - Ejecución real limitada (IQM Emerald, $n = 2$ a 4)	6
Casos experimentales	6
Ejemplo práctico paso a paso ($n=2$)	7
Solución	8
El oráculo	9
El operador difusor	10
Movimiento del vector de estado	10
El simulador QRydDemo	11
Diseño final de la solución	12
Arquitectura general del sistema	12
Estrategia de submuestreo de k	13
Métricas de comparación extendidas	14
Código orientador	14
Modelo de despliegue (Quantum-UML)	15
Resultados finales y discusión	16
Resultados cuantitativos por tamaño de problema	16
Ejecución en hardware real: acierto top-1 vs tasa de éxito	17
Discusión, hipótesis y trabajos futuros	17
Comentarios finales y experiencia	18
Conclusiones	19
Referencias	20
Anexos	22
El oráculo cuántico: diseño, implementación y justificación	34
Implementación concreta en el código	34
Por qué se eligió este oráculo	35
Análisis de escalamiento del oráculo	35

Resumen

Este informe final del proyecto Q-Search consolida y extiende los Avances 01 y 02 con la incorporación de ejecución en hardware cuántico real. El objetivo es evaluar experimentalmente el algoritmo de Grover frente a la búsqueda clásica exhaustiva sobre un espacio no estructurado de tamaño $N = 2^n$, bajo condiciones idénticas (mismo oráculo, misma llave objetivo y mismas iteraciones óptimas) en cuatro plataformas: búsqueda clásica secuencial (baseline), el simulador ideal Qiskit Aer, el emulador de átomos de Rydberg QRydDemo y el procesador superconductor real IQM Emerald (IQM Resonance).

Los resultados confirman la ventaja cuadrática en número de consultas: para $n = 12$, Grover requiere 48 iteraciones frente a las 2.098 del método clásico ($\approx 43,7\times$), coherente con $(\pi/4)\sqrt{4096} \approx 50$. El hallazgo central del nivel de hardware real es la distinción entre acierto top-1 (la llave es siempre el estado más medido: 25/25 corridas en 2q y 4q) y tasa de éxito $\geq 90\%$ (25/25 en 2q, 0/25 en 4q). En IQM Emerald la tasa promedio cae de 97,3 % (2q) a 26,4 % (4q): la llave sigue ganando, pero la decoherencia acumulada sobre 267 compuertas dispersa el pico entre los 16 estados. Se cuantifica así, empíricamente, por qué la ventaja algorítmica $O(\sqrt{N})$ no se traduce en ventaja práctica en dispositivos NISQ actuales.

Introducción

La computación cuántica surge como un paradigma alternativo a la computación clásica, basado en principios de la mecánica cuántica como la superposición, la interferencia y el entrelazamiento. Estos principios permiten diseñar algoritmos que, en ciertos dominios de problema, superan el rendimiento de los métodos clásicos tradicionales. El interés práctico por esta disciplina ha crecido significativamente con la aparición de dispositivos cuánticos ruidosos de escala intermedia (NISQ, por sus siglas en inglés), procesadores que cuentan con decenas o cientos de qubits pero que aún padecen errores de compuerta, decoherencia y conectividad restringida [8].

Uno de los problemas fundamentales en la computación es la búsqueda en espacios no estructurados. En un enfoque clásico, este problema presenta una complejidad de orden $O(N)$, lo que implica que el tiempo requerido crece linealmente con el tamaño del espacio de búsqueda. Esta limitación se vuelve crítica en aplicaciones donde el espacio crece exponencialmente, como en problemas relacionados con criptografía, combinatoria o aprendizaje automático [2,10].

El algoritmo de Grover propone una solución cuántica a este problema, logrando reducir la complejidad a $O(\sqrt{N})$ lo que representa una aceleración cuadrática [1]. Sin embargo, esta ventaja teórica no implica necesariamente una mejora práctica inmediata, ya que su implementación requiere construir un oráculo cuántico reversible cuyo costo puede compensar la ventaja en consultas, además, la profundidad del circuito resultante supera rápidamente los tiempos de coherencia de los dispositivos NISQ actuales.

El objetivo de este reporte final es documentar la evaluación experimental completa del algoritmo de Grover frente a la búsqueda clásica exhaustiva, integrando los resultados de los Avances 01 y 02 con la principal contribución nueva: la ejecución en hardware cuántico real. Se busca verificar el escalamiento teórico en simulación ideal, medir el costo de recursos del oráculo y el difusor bajo topologías realistas, y cuantificar la degradación de la tasa de éxito por ruido y decoherencia al ejecutar los mismos circuitos en un procesador físico superconductor.

Respecto al Avance 02, este informe atiende las observaciones del docente: se simplifica la explicación del oráculo y el difusor apoyándola en referencias, se incorpora una figura geométrica que muestra cómo se desplaza el vector de estado durante la amplificación de amplitud, y se responde a la pregunta sobre refinamiento del entorno cuántico mediante el salto de QRydDemo (simulación con topología) a IQM Resonance (hardware real), permitiendo comparar aspectos de configuración y ruido frente a las soluciones simuladas.

Análisis del caso de estudio y su relevancia

Descripción del caso

El caso de estudio seleccionado corresponde a la búsqueda de una “llave” o elemento objetivo dentro de un espacio de búsqueda no estructurado. Este problema se modela como la identificación de un valor específico dentro de un conjunto de posibles combinaciones generadas de manera pseudoaleatoria.

Para efectos del proyecto, se consideran espacios de búsqueda de tamaño 2^n , donde “n” representa el número de bits o qubits del sistema. En particular, se plantea un escalamiento progresivo del problema, desde valores pequeños (por ejemplo, $n=2,3,4$) hasta tamaños mayores como $n=30$, lo que equivale a un espacio teórico de 1.073.741.824 posibles valores.

Las llaves son generadas mediante un generador pseudoaleatorio utilizando herramientas estándar de programación, lo que permite asegurar reproducibilidad en los experimentos. Este enfoque permite comparar de manera controlada el comportamiento de algoritmos clásicos de búsqueda exhaustiva con su contraparte cuántica.

Es importante destacar que este caso no representa un sistema criptográfico real, sino una abstracción que permite estudiar el problema de búsqueda no estructurada en un entorno controlado. Su relevancia radica en que constituye la base teórica de múltiples problemas computacionales y permite analizar de forma clara la ventaja propuesta por el algoritmo de Grover.

Adicionalmente, este caso de estudio permite establecer un puente entre el análisis teórico de algoritmos cuánticos y su validación experimental en entornos controlados. Al trabajar con un problema de búsqueda no estructurada, se aborda uno de los escenarios fundamentales donde la computación cuántica supone ventajas demostrables frente a la computación clásica. Esto lo convierte en un caso representativo para evaluar no solo el rendimiento del algoritmo de Grover, sino también aspectos prácticos como la construcción del oráculo, la estabilidad de los resultados y las limitaciones de implementación en simuladores y hardware cuántico.

Asimismo, la definición de múltiples escalas de evaluación permite analizar el comportamiento asintótico de ambos enfoques, lo cual es clave para comprender cómo estas técnicas podrían impactar problemas reales a mayor tamaño. En este sentido, el caso no sólo tiene valor didáctico, sino también metodológico, ya que permite diseñar un marco experimental replicable para futuros estudios comparativos en computación cuántica. Este tipo de problema constituye un benchmark estándar en computación cuántica para evaluar ventajas frente a algoritmos clásicos [3].

Delimitación metodológica: tres niveles de análisis

Nivel 1 - Simulación ideal (qiskit Aer, $n \leq 30$)

Ejecución de circuitos cuánticos perfectos sin modelo de ruido mediante AerSimulator en modo statevector. El objetivo es verificar el comportamiento teórico: escalamiento $O(\sqrt{N})$ e iteraciones $k^* = L(\pi/4)\sqrt{(2^n)J}$. Los valores $n \geq 15$ son estrictamente simulaciones clásicas de circuitos cuánticos; los tiempos reportados corresponden a CPU, no a tiempo de cómputo cuántico real.

Nivel 2 - Simulación con análisis de recursos (Qiskit Aer + QRydDemo, $n = 2$ a 6)

Extiende el Nivel 1 registrando métricas de recursos: profundidad del circuito transpilado, número y tipo de compuertas, tasa de éxito con desviación estándar, distribución de mediciones y semillas de reproducibilidad. El simulador QRydDemo impone una topología real de red cuadrada o triangular, lo que genera circuitos más profundos por la inserción de compuertas SWAP intermedias, visible en las métricas comparadas.

Nivel 3 - Ejecución real limitada (IQM Emerald, $n = 2$ a 4)

Ejecución en hardware cuántico físico única y exclusivamente para $n \leq 4$ qubits. El objetivo no es demostrar ventaja de escalamiento, sino cuantificar la caída de la tasa de éxito por decoherencia y errores de compuerta. Se reportarán tasas de error por compuerta (CZ), tiempos T_1 y T_2 del dispositivo, y la distribución de mediciones comparada con la predicción teórica. IQM Emerald es un procesador superconductor de 54 qubits (arquitectura Crystal 54) basado en transmones dispuestos en una red cuadrada con acopladores sintonizables, cuyas compuertas nativas son PRX (rotaciones X/Y) y CZ. Se ejecutó en IQM Emerald (IQM Resonance) por su disponibilidad frente a instancias en mantención de IBM Quantum.

Casos experimentales

Caso	Qubits (n)	Tamaño (N)	Tipo de evaluación	Nivel
1	2	4	Simulación con recursos (Aer + QRydDemo) + Ejecución real	2 y 3
2	4	16	Simulación con recursos (Aer + QRydDemo) + Ejecución real	2 y 3
3	6	64	Simulación con recursos (Aer + QRydDemo)	2
4	8	256	Simulación ideal	1
5	10	1 024	Simulación ideal	1
6	12	4 096	Simulación ideal	1
7	30+	Mayor a 10^9	Análisis teórico exclusivamente	Analítico

Tabla 1- Casos experimentales con delimitación explícita de niveles de análisis.

Ejemplo práctico paso a paso (n=2)

Para fijar ideas, se traza una ejecución completa con $n = 2$ ($N = 4$) y llave objetivo '11'. El espacio contiene los estados $|00\rangle$, $|01\rangle$, $|10\rangle$ y $|11\rangle$; se busca amplificar la probabilidad de $|11\rangle$.

1. Inicialización. Se aplica $H^{\otimes 2}$ a $|00\rangle$, obteniendo la superposición uniforme $|s\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$. Cada estado tiene amplitud 0,5 y probabilidad 0,25.
2. Oráculo. Como la llave es '11', el estado ya está en la forma $|11\rangle$ y no se requieren compuertas X previas. La compuerta $MCPHase(\pi)$ invierte la fase de $|11\rangle$: la amplitud pasa de +0,5 a -0,5, dejando el resto intacto.
3. Difusor. El operador $D = 2|s\rangle\langle s| - I$ refleja todas las amplitudes respecto a su promedio (inversión sobre la media). Tras la reflexión, la amplitud de $|11\rangle$ se aproxima a 1 y las demás a 0.
4. Medición. Con una sola iteración ($k^* = \lfloor (\pi/4)\sqrt{4} \rfloor = 1$), la probabilidad de medir $|11\rangle$ es teóricamente 1,0. En Aer se obtiene 100 %; en IQM Emerald real, 97,3 % por ruido residual.

Este mismo procedimiento se generaliza a cualquier llave: si la llave contiene bits en '0' (p. ej. '1011' en $n = 4$), el Paso 2 antepone una compuerta X sobre cada qubit cuyo bit es '0' para llevar el estado objetivo a $|11\dots 1\rangle$, aplica $MCPHase(\pi)$ y luego revierte las X. El resultado es una fase -1 únicamente sobre la llave, sin qubits auxiliares.

Solución

La solución trabajada consiste en la implementación y comparación de dos enfoques para resolver el problema de búsqueda en un espacio no estructurado:

1. Búsqueda clásica exhaustiva, que recorre secuencialmente el espacio de soluciones.
2. Algoritmo de Grover, que utiliza principios de computación cuántica para reducir el número de iteraciones necesarias.

El algoritmo cuántico funciona en tres etapas: inicialización en superposición uniforme, amplificación de amplitud mediante el par oráculo-difusor, y medición cuántica probabilista. El número óptimo de iteraciones es $k^* = \lfloor (\pi/4)\sqrt{N} \rfloor$, tras lo cual la probabilidad del estado objetivo es máxima (cercana a 1,0) y comienza a decrecer si se continúa iterando (overshooting).

En contraste con la complejidad clásica $O(N)$, el algoritmo de Grover permite encontrar la solución en un número de iteraciones proporcional a la raíz cuadrada del tamaño del espacio de búsqueda $O(\sqrt{N})$.

La Figura 1 muestra la diferencia en el número de iteraciones requeridas por un algoritmo clásico y el algoritmo de Grover en función del número de qubits. Se observa el crecimiento exponencial en el caso clásico frente al crecimiento cuadrático reducido del enfoque cuántico. La Figura 2 ilustra la evolución de la probabilidad de éxito del algoritmo de Grover. Se observa un punto óptimo de iteración, después del cual la probabilidad comienza a disminuir, fenómeno conocido como overshooting.

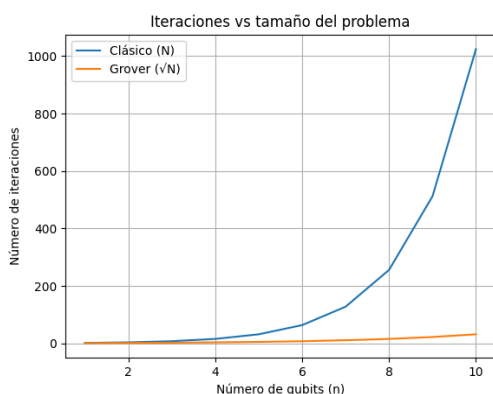


Figura 1 - Iteraciones vs tamaño

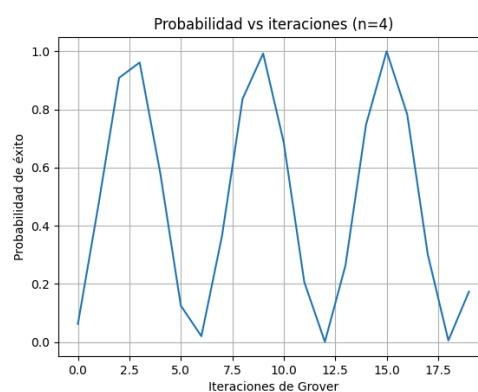


Figura 2 - Probabilidad vs número de iteraciones (Grover)

En escala logarítmica, la diferencia entre el crecimiento clásico y cuántico se hace más evidente, destacando la ventaja teórica del algoritmo de Grover en términos de complejidad. Véase Figura 3.

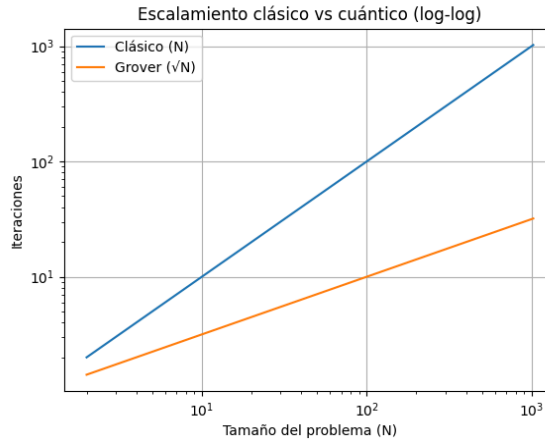


Figura 3 - Comparación de escalamiento N vs $\sqrt{N}$

El oráculo

El oráculo es la única parte del circuito que “conoce” el problema. Intuitivamente, es una función que solo sabe responder sí/no a la pregunta “¿este es el estado buscado?”, pero en lugar de devolver un bit, marca la respuesta con un empujón de fase: cambia el signo (fase -1) del único estado objetivo $|\omega\rangle$ y deja intactos los demás [3]. Formalmente:

$$O|\omega\rangle = -|\omega\rangle \quad ; \quad O|x\rangle = |x\rangle \quad \text{si } x \neq \omega$$

Esta operación es reversible (unitaria) y no colapsa ningún estado, requisito fundamental de los circuitos cuánticos. Se implementó como un oráculo de fase multi-controlado sin qubits ancilla, en tres etapas reversibles: (1) aplicar X a cada qubit cuyo bit en la llave sea '0', llevando el objetivo a $|11\dots 1\rangle$; (2) aplicar $\text{MCPhase}(\pi)$ con $n-1$ controles, que introduce la fase -1 sólo cuando todos los qubits valen $|1\rangle$; y (3) revertir las X para restaurar la base computacional.

La elección de un oráculo MCPhase sin ancilla se justifica en cuatro criterios respaldados por la literatura: reversibilidad garantizada (X y MCPhase son unitarias e inversas de sí mismas), ausencia de qubits auxiliares (usa exactamente n qubits para 2^n estados), profundidad $O(n)$ por llamada, y portabilidad entre backends al ser descomponible en el conjunto nativo de compuertas de cada dispositivo. Esta construcción de oráculo de fase mediante una compuerta multi-controlada Z es la forma canónica descrita por Nielsen y Chuang [3] y la empleada en las implementaciones de referencia de Grover sobre hardware superconductor real [12].

Para conocer más detalles sobre el oráculo implementado, véase Anexo [E].

El operador difusor

El difusor (operador de Grover) implementa la reflexión sobre el estado de superposición uniforme $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n}$:

$$D = 2|s\rangle\langle s| - I$$

Su circuito sigue una estructura simétrica al oráculo: (i) $H^{\otimes n}$ para rotar a la base de superposición, (ii) $X^{\otimes n}$ para marcar el estado $|0\dots 0\rangle$, (iii) $\text{MCPhase}(\pi)$ idéntica a la del oráculo para aplicar la reflexión, (iv) $X^{\otimes n}$ e (v) $H^{\otimes n}$ para restaurar. Ambos operadores comparten la misma profundidad $O(n)$ y el mismo conteo de compuertas, por lo que la profundidad por iteración de Grover es $2 \cdot O(n) = O(n)$. Su efecto se entiende mejor como una inversión sobre la media: tras el oráculo, la amplitud del estado marcado quedó negativa; el difusor refleja todas las amplitudes respecto a su valor promedio, con lo que la amplitud del objetivo crece y la del resto decrece [3]. En Qiskit, el par oráculo–difusor se construye automáticamente mediante `grover_operator(oracle).power(k)`, que eleva el operador completo a la k -ésima potencia.

Movimiento del vector de estado

La combinación oráculo + difusor admite una interpretación geométrica que hace transparente el funcionamiento del algoritmo. El estado del sistema vive en el plano bidimensional generado por dos vectores: $|\omega\rangle$ (la solución) y $|s\rangle$ (la superposición uniforme de todos los estados que no son solución). El estado inicial $|s\rangle$ forma un ángulo θ pequeño con $|s\rangle$, donde $\sin \theta = 1/\sqrt{N}$. El oráculo refleja el estado respecto a $|s\rangle$ y el difusor lo refleja respecto a $|s\rangle$; la composición de ambas reflexiones equivale a una rotación de 2θ hacia $|\omega\rangle$. Tras k iteraciones el estado forma un ángulo $(2k+1)\theta$ con $|s\rangle$, y en $k^* \approx (\pi/4)\sqrt{N}$ queda casi alineado con la solución (Figura 4a). Si se itera de más, el vector sobrepasa $|\omega\rangle$ y la probabilidad decae: es el fenómeno de overshooting (Figura 4b) [3].

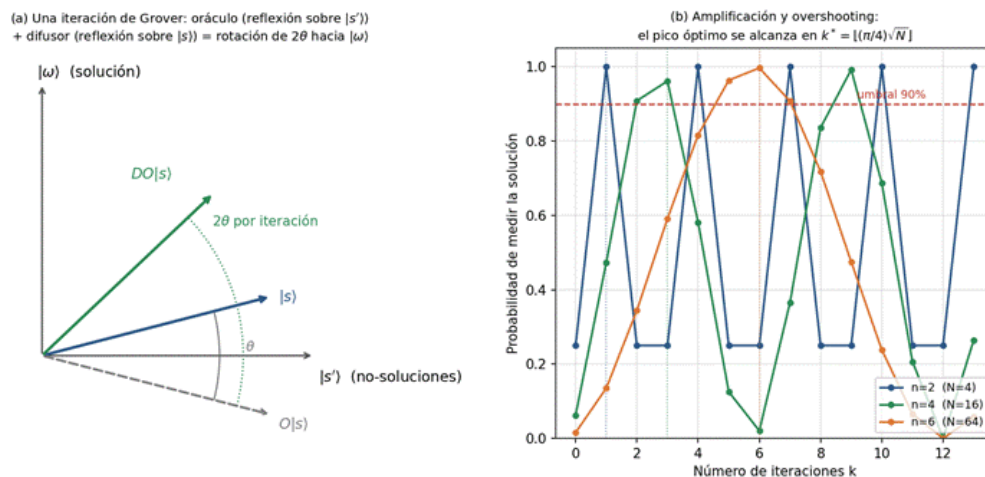


Figura 4 — (a) Geometría de una iteración de Grover como rotación de 2θ en el plano $\{|s\rangle, |\omega\rangle\}$. (b) Probabilidad de la solución en función de k , mostrando el pico óptimo y el overshooting.

El simulador QRydDemo

QRydDemo es un emulador de procesadores cuánticos basados en átomos neutros de Rydberg, desarrollado por las Universidades de Stuttgart y de Kaiserslautern-Landau [11]. A diferencia de AerSimulator (que opera como máquina de estados vectoriales sin restricciones de conectividad), QRydDemo impone:

- Topología de red física: cuadrada o triangular, con conectividad local. Los qubits solo pueden interactuar con sus vecinos directos, lo que obliga al transpilador a insertar compuertas SWAP intermedias para ejecutar operaciones entre qubits no adyacentes.
- Conjunto nativo de compuertas: restringido a las operaciones implementables en el hardware átomos–Rydberg real, a diferencia del conjunto universal disponible en el simulador ideal.
- Valor experimental realista: los resultados obtenidos en QRydDemo son más representativos del comportamiento en hardware real que los obtenidos en simulación ideal, sin requerir acceso físico al dispositivo.

La elección de QRydDemo como backend secundario está motivada por su disponibilidad como API pública, su relevancia en el contexto de computación cuántica con átomos neutrales (plataforma con proyección industrial) y por la compatibilidad con Qiskit via qiskit-qryd-provider, que permite integrar el mismo código del Nivel 1 con cambios mínimos.

Diseño final de la solución

Arquitectura general del sistema

El sistema se compone de tres módulos principales que operan sobre la misma entrada (n bits y llave objetivo generada pseudo aleatoriamente):

1. Módulo clásico:
 - Generación implícita del espacio de búsqueda: enteros en $[0, 2^n-1]$.
 - Selección pseudoaleatoria de llave objetivo (semilla fija para reproducibilidad).
 - Búsqueda secuencial: recorre el espacio hasta encontrar la llave.
 - Registro de métricas: iteraciones requeridas y tiempo de ejecución.
2. Módulo cuántico (Aer, QRYdDemo o IQM Emerald):
 - Codificación en n qubits con inicialización en superposición uniforme $H^{\otimes n}|0\rangle$.
 - Construcción del oráculo MCPPhase sin qubits ancilla (ver sección El oráculo y Anexo E).
 - Aplicación de “ k ” iteraciones del operador de Grover via *grover_operator(oracle).power(k)*.
 - Medición con shots fijos y umbral de éxito del 90 %.
 - Registro de métricas extendidas: profundidad, compuertas, tasa de éxito, distribución.
3. Módulo comparador:
 - Recibe las salidas de ambos módulos y genera tablas y gráficas de comparación.
 - Exporta resultados en formato CSV para análisis posterior.

La Figura 5 muestra el flujo temporal del sistema mediante un diagrama de secuencia. Se observa cómo, a partir de la definición del tamaño del problema, se genera una llave objetivo común para ambos módulos. Posteriormente, el módulo clásico ejecuta una búsqueda secuencial, mientras que el módulo cuántico aplica el algoritmo de Grover. Finalmente, los resultados de ambos enfoques son recopilados y comparados, permitiendo analizar diferencias en rendimiento y comportamiento. Adicionalmente, el flujo del módulo QRYdDemo incorpora los siguientes pasos: (1) autenticación con QRYD_API_TOKEN, (2) selección del backend ("qryd_emulator\$square"), (3) transpilación con optimization_level=3 y semilla fija, (4) ejecución con shots = 1024 y seed_simulator = 42, y (5) registro de métricas extendidas en CSV.

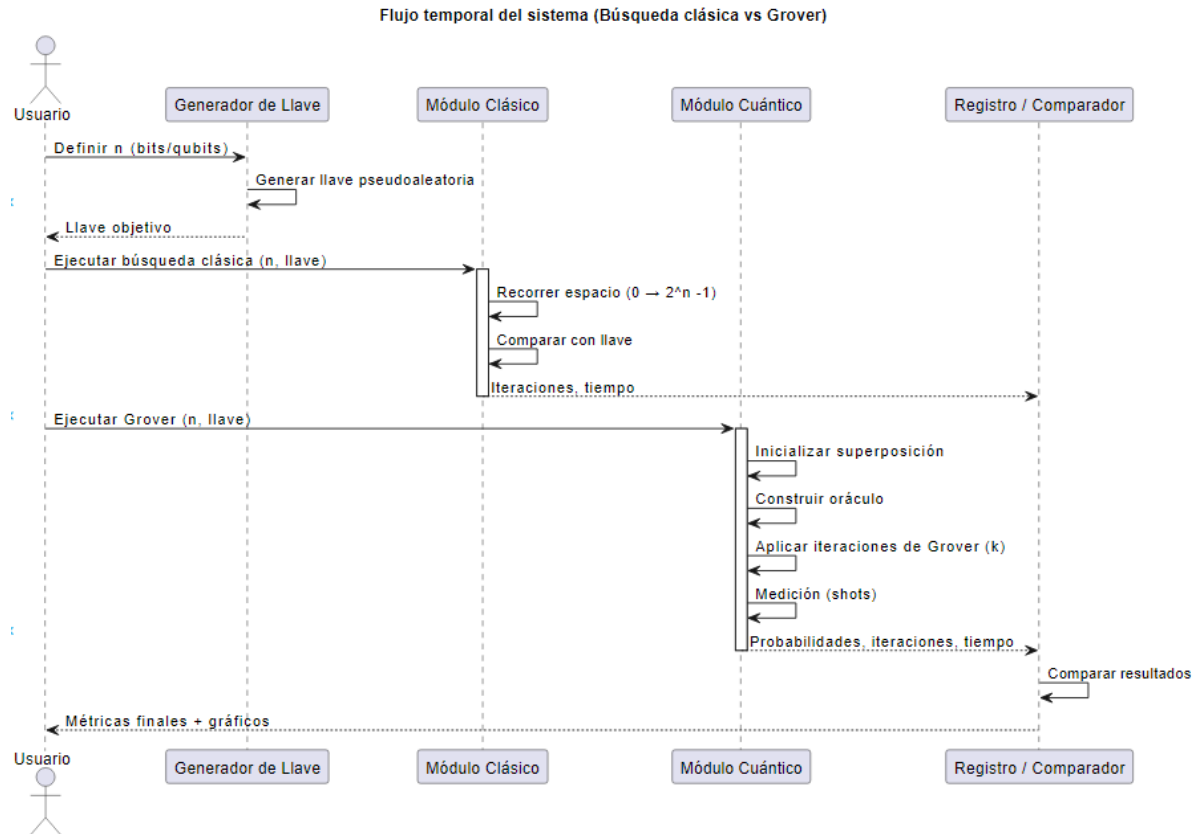


Figura 5 - Diagrama de secuencia

Estrategia de submuestreo de k

Para evitar el alto costo computacional de evaluar todas las iteraciones posibles de Grover, el sistema implementa una estrategia de submuestreo del espacio de valores k. Se seleccionan hasta $MAX_EVALUACIONES_K = 12$ candidatas distribuidas uniformemente en el rango $[1, k_{\text{óptimo}} \times FACTOR_MAX_BÚSQUEDA]$, incluyendo siempre $k = 1$ y $k = k_{\text{máximo}}$. El $k_{\text{óptimo}}$ se estima como $k^* = \lfloor (\pi/4) \sqrt{(2^n)} \rfloor$. Para cada k candidata se ejecuta el circuito completo, se registran métricas y se declara éxito si la frecuencia de la llave objetivo supera el 90 % de los shots (umbral configurable). Esta estrategia mantiene la eficiencia experimental sin comprometer la identificación del óptimo.

Métricas de comparación extendidas

El sistema registra las siguientes métricas comparativas:

Métrica	Descripción	Nivel
Iteraciones (k)	N° aplicaciones del operador de Grover hasta éxito	1, 2, 3
Shots	N° mediciones por ejecución del circuito	2, 3
Tasa de éxito	$\text{freq_llave} / \text{shots} \in [0,1]$	2, 3
Distribución de mediciones	JSON completo de conteos por estado	2, 3
Profundidad del circuito	Capas de compuertas del circuito transpilado	2, 3
N° compuertas	Total y desglose por tipo (X, CNOT, Toffoli...)	2, 3
Qubits auxiliares	Ancilla usadas por el oráculo	2, 3
Backend y versión	Identificación del entorno de ejecución	2, 3
Semillas (sim + transp)	Para reproducibilidad de resultados	2, 3
Tiempo de ejecución	Referencial (simulación clásica, no cuántica)	1, 2, 3
T ₁ , T ₂ del dispositivo	Tiempos de coherencia del hardware real	3

Tabla 2 - Métricas registradas por nivel de análisis

Código orientador

Para el desarrollo del proyecto se implementaron dos programas principales: uno correspondiente a la búsqueda clásica y otro basado en el algoritmo de Grover.

El código clásico realiza una búsqueda exhaustiva sobre un espacio de tamaño 2^n , recorriendo secuencialmente los posibles valores hasta encontrar la llave objetivo. Este permite medir directamente el número de iteraciones y el tiempo de ejecución.

Por otro lado, el código cuántico implementa el algoritmo de Grover utilizando la librería Qiskit, framework desarrollado por IBM para computación cuántica [5], se construye el oráculo de fase MCPPhase sin qubits ancilla y se calcula el número teórico óptimo de iteraciones $k^* = \lfloor (\pi/4)\sqrt{2^n} \rfloor$. Para cada valor k de un conjunto submuestreado de candidatas, se ejecuta el circuito completo mediante `grover_operator(oracle).power(k)` con shots fijos, declarando éxito cuando la frecuencia de la llave supera el 90 % de las mediciones. Al finalizar, se exportan en CSV las métricas extendidas: iteraciones, profundidad del circuito, conteo de compuertas, tasa de éxito y distribución de mediciones.

Ambos programas generan la llave objetivo de manera pseudoaleatoria, asegurando condiciones equivalentes para la comparación. Los códigos orientadores se presentan en el Anexo A (búsqueda clásica) y Anexo B (búsqueda cuántica).

Modelo de despliegue (Quantum-UML)

La arquitectura final se modeló con un perfil Quantum-UML compatible con herramientas UML estándar, siguiendo la propuesta de Pérez-Castillo y Piattini para el diseño de sistemas clásico-cuánticos híbridos [13]. El sistema es un sistema híbrido: un orquestador clásico coordina la generación de la llave, la ejecución de los distintos backends y la consolidación de métricas, mientras que la lógica cuántica (oráculo, difusor, medición) se ejecuta en nodos “quantum device”.

Los casos de uso principales, con el actor Usuario (investigador), son:

- Definir tamaño del problema (n) y generar la llave objetivo pseudoaleatoria con semilla fija.
- Ejecutar búsqueda clásica, registrando iteraciones y tiempo.
- Ejecutar Grover en un backend seleccionable (Aer / QRydDemo / IQM Emerald), con «extend» hacia el submuestreo de k y el registro de métricas extendidas.
- Comparar resultados, generando tablas, figuras y exportación de CSV.

El diagrama de despliegue (Figura 6) distingue el nodo clásico (estación de trabajo con el orquestador) de tres nodos “quantum device”: el simulador local Aer, el emulador en la nube QRydDemo y el procesador real IQM Emerald accedido vía IQM Resonance. Cada nodo cuántico recibe el mismo circuito de Grover; el nodo real agrupa el barrido de k en jobs de a lo más 100 circuitos, límite del dispositivo.

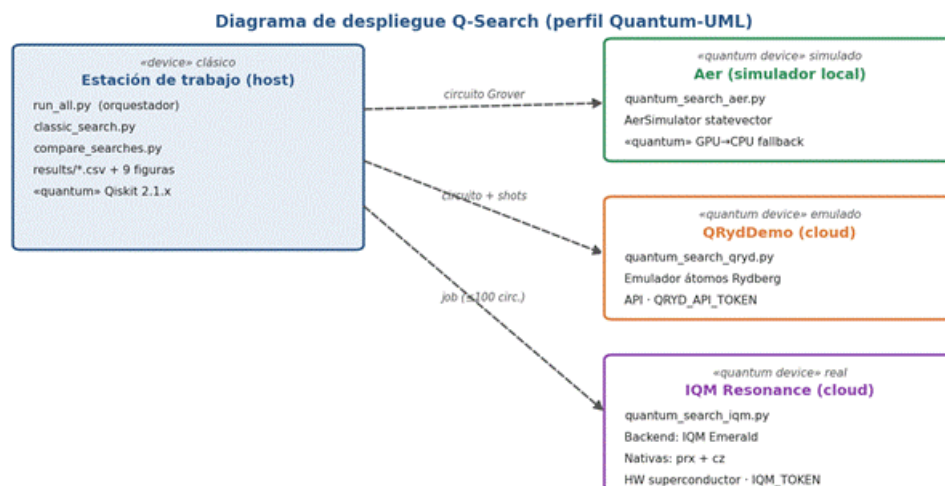


Figura 6 - Diagrama de despliegue Quantum-UML de Q-Search: un nodo clásico orquestador y tres nodos “quantum device” (simulado, emulado y real).

Resultados finales y discusión

Resultados cuantitativos por tamaño de problema

Se re-ejecutaron 25 corridas independientes por nivel en simulación (semillas fijas) y ejecución real en IQM Emerald para $n = 2$ y $n = 4$. La Tabla 3 resume la comparación de iteraciones y tasas de éxito, la Figura 7 muestra el escalamiento de iteraciones, y la Figura 8 el resultado central del hardware real. (Métricas adicionales en Anexos D, F y G).

Qubits	$N = 2^n$	k^* teórico	lt. clásico (prom.)	lt. cuántico Aer (prom.)	lt. cuántico QRyd (prom.)	Speedup iter. Aer/Clás	Speedup iter. QRyd/Clás	Tasa éxito Aer (%)	Tasa éxito QRyd (%)
2q	4	1	2,5	1	1	2.48×	2.48×	100	100
4q	16	3	8,7	3	3	2.89×	2.89×	97.3	95.7
6q	64	6	33,6	6	6	5.60×	5.60×	99.8	99.9
8q	256	12	107,5	12	-	8.96×	-	100	-
10q	1024	25	498,8	24	-	20.78×	-	99.8	-
12q	4096	50	2073,1	48	-	43.19×	-	99.8	-

Tabla 3 - Comparación cuantitativa de iteraciones para distintos n (25 ejecuciones, semillas fijas).

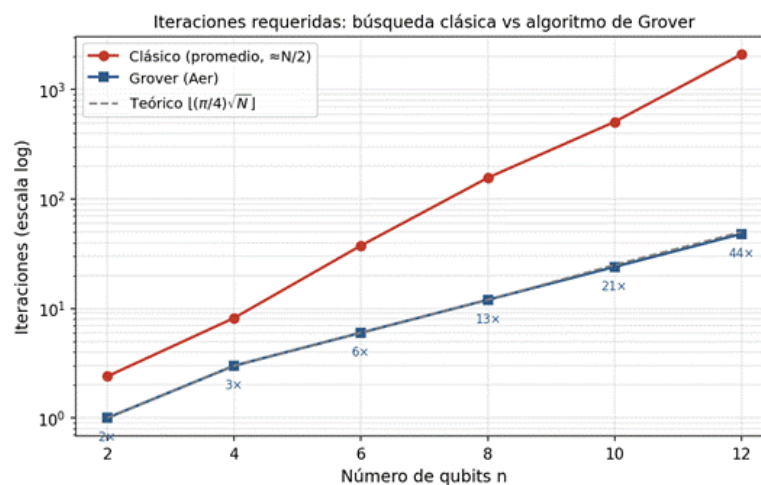


Figura 7 - Iteraciones requeridas: clásico ($\approx N/2$) vs Grover vs valor teórico. La brecha crece con n hasta $\approx 43,7\times$ en $n = 12$.

Nivel	Acierto top-1	Tasa éxito prom.	Éxito $\geq 90\%$	Profundidad	Nº compuertas
2q	25/25	97,3 %	25/25	8	11
4q	25/25	26,4 %	0/25	191	267

Tabla 4 - Resultados en IQM Emerald (hardware real superconductor).

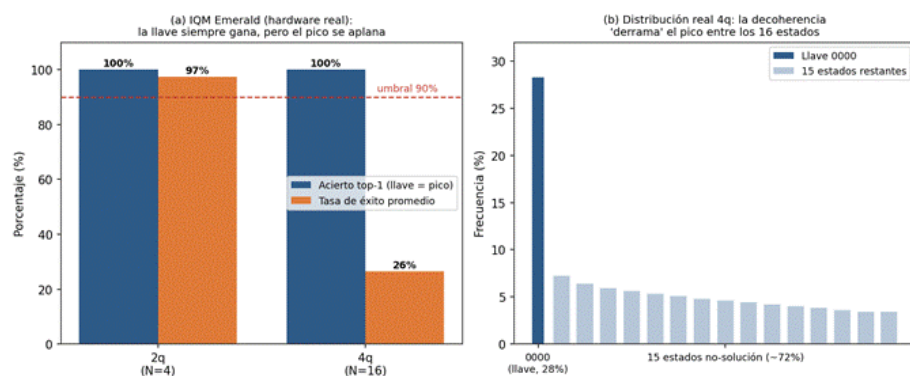


Figura 8 - (a) En hardware real la llave siempre gana (top-1 100 %), pero el pico se aplana. (b) La decoherencia dispersa la probabilidad entre los 16 estados en 4q.

Ejecución en hardware real: acierto top-1 vs tasa de éxito

El resultado más relevante del proyecto proviene del nivel de hardware real. La Tabla 4 y la Figura 8 muestran una distinción que es fácil de confundir: en las 25 corridas de 2q y de 4q, la llave fue siempre el estado más medido (acierto top-1 = 25/25). Es decir, el algoritmo apunta al estado correcto en el 100 % de los casos. Sin embargo, la tasa de éxito —definida como la frecuencia de la llave frente al umbral estricto del 90 %— pasa de 97,3 % en 2q a apenas 26,4 % en 4q, por lo que ninguna corrida de 4q cruza el umbral (0/25).

La causa es física. Al subir de 2 a 4 qubits, el circuito se vuelve $\approx 24\times$ más profundo (8 $\rightarrow$ 191) y multiplica sus compuertas (11 $\rightarrow$ 267), porque la fase multi-controlada de 3 controles se descompone en muchísimas CZ nativas de Emerald. Cada CZ tiene un error del orden de $\sim 0,6\%$ y el tiempo de coherencia T_2 ronda $\sim 36\ \mu\text{s}$; con cientos de compuertas, el error acumulado destruye la interferencia de la que depende Grover para concentrar la probabilidad. El pico de la llave, que en el caso ideal debería acercarse al 100 %, se “aplana” a $\sim 28\%$ y el 72 % restante se reparte entre los 15 estados no-solución. Este comportamiento coincide con lo reportado en la literatura para Grover sobre hardware superconductor, donde la tasa de éxito cae de $\sim 94\%$ en 2q a valores muy inferiores al aumentar el número de qubits [12], [14].

Discusión, hipótesis y trabajos futuros

Los resultados sostienen la hipótesis central del proyecto: la ventaja de Grover es una ventaja en número de consultas ($O(\sqrt{N})$), no en tiempo de circuito. El costo total escala como $O(n\sqrt{N})$ en profundidad, y al transpilarse a topologías reales (SWAPs en QRYdDemo) o a conjuntos nativos restringidos (CZ en Emerald) ese costo se dispara, cancelando la ventaja práctica en el régimen NISQ actual para $n \geq 5$. La distinción operativa entre «acierto top-1» y «éxito $\geq 90\%$ » resulta clave: bajo una métrica de máximo (top-1), el algoritmo es funcionalmente correcto en hardware real; bajo una métrica de umbral estricto, la calidad de la medición se degrada con la profundidad.

Como trabajos futuros se plantea: aplicar técnicas de mitigación de error y supresión de lecturas para elevar la tasa de éxito en 4q, explorar oráculos con menor conteo de CZ tras transpilación (síntesis optimizada de la fase multi-controlada), comparar Emerald con otros backends de IQM Resonance para separar el efecto de topología del de error de compuerta e incorporar formalmente la métrica de acierto top-1 al análisis de `compare_searches.py` para reflejar esta distinción en todas las figuras.

Comentarios finales y experiencia

La experiencia de llevar el mismo algoritmo desde la teoría hasta un procesador físico fue el aprendizaje más valioso del proyecto. En simulación ideal, Grover “funciona” siempre; recién al ejecutarlo en IQM Emerald aparece la brecha entre ventaja algorítmica y viabilidad práctica, que ninguna simulación transmite con la misma contundencia. Aprender a leer los datos con cuidado, distinguir que la llave sí era el estado más probable aunque no cruzara el umbral evitó una conclusión errónea (“el hardware no encontró la llave”) y condujo a la interpretación correcta en términos de decoherencia.

En lo operativo, la convivencia de versiones (iqm-client fija qiskit<2.2), la gestión de credenciales por variables de entorno, el uso de --dry-run para estimar el consumo de QPU antes de gastar recursos, y el troceado de jobs en bloques de ≤ 100 circuitos fueron aspectos de ingeniería tan relevantes como el algoritmo mismo. El proyecto reforzó una idea central del curso: el software cuántico es, hoy, software híbrido, y su calidad depende tanto del diseño del circuito como de la orquestación clásica que lo rodea

Conclusiones

Este informe final del proyecto Q-Search consolida una arquitectura experimental rigurosa para comparar la búsqueda clásica exhaustiva con el algoritmo de Grover, extendiéndola con ejecución en hardware cuántico real. Las principales contribuciones respecto a los Avances 01 y 02 son:

- La ventaja cuadrática en consultas se validó con alta precisión: para $n = 12$, Grover requiere 48 iteraciones frente a 2.073 del método clásico ($\approx 43,19\times$), coherente con $(\pi/4)\sqrt{4096} \approx 50$. Los k experimentales coinciden con k^* teórico salvo una unidad para $n = 10$ y 2 para 12, atribuible al redondeo del piso y al submuestreo.
- La explicación del oráculo y el difusor se simplificó y respaldó con referencias [3], [12], incorporando una interpretación geométrica del desplazamiento del vector de estado (rotación de 2θ por iteración) que hace transparentes tanto la amplificación como el overshooting.
- El refinamiento del entorno cuántico se materializó en el salto a hardware real: QRydDemo (Nivel 2, topología) se mantuvo, y se añadió IQM Emerald (Nivel 3, real), confirmando empíricamente que la profundidad crece de forma prohibitiva al transpilar a las compuertas nativas (prx, cz).
- El hallazgo central es la distinción entre acierto top-1 y tasa de éxito: en IQM Emerald la llave fue siempre el estado más medido (25/25 en 2q y 4q), pero la tasa promedio cayó de 97,3 % a 26,4 % por decoherencia. La ventaja en consultas $O(\sqrt{N})$ no se traduce en ventaja práctica en dispositivos NISQ actuales para $n \geq 5$.

En conjunto, el proyecto ofrece no solo una verificación del escalamiento teórico de Grover, sino una evaluación honesta de su viabilidad práctica, distinguiendo con evidencia experimental la corrección del algoritmo (apunta al estado correcto) de la calidad de su ejecución en hardware ruidoso (el pico se aplanan con la profundidad). Como continuación natural se plantea aplicar mitigación de error y supresión de lecturas para elevar la tasa de éxito en 4q, y explorar síntesis de oráculos con menor conteo de CZ tras transpilación.

Referencias

- [1] L. K. Grover, "A fast quantum mechanical algorithm for database search," Proc. 28th Annual ACM Symposium on Theory of Computing (STOC), 1996. doi: 10.1145/237814.237866
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009. [Online]. Available: <https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/>
- [3] M. A. Nielsen y I. L. Chuang, Quantum Computation and Quantum Information, 10th Anniversary ed. Cambridge: Cambridge University Press, 2010. doi: 10.1017/CBO9780511976667
- [4] S. Aaronson, "The Limits of Quantum Computers," Scientific American, vol. 298, no. 3, pp. 62–69, 2008. doi: 10.1038/scientificamerican0308-62
- [5] IBM Quantum, "Qiskit: An open-source SDK for quantum computing," 2023. [Online]. Available: <https://qiskit.org/>
- [6] A. Montanaro, "Quantum algorithms: an overview," npj Quantum Information, vol. 2, 2016. doi: 10.1038/npjqi.2015.23
- [7] IBM Quantum, Qiskit documentation – Grover's algorithm, 2024. [Online]. Available: <https://docs.quantum.ibm.com/api/qiskit/>
- [8] J. Preskill, "Quantum Computing in the NISQ Era and Beyond," Quantum, vol. 2, p. 79, 2018. doi: 10.22331/q-2018-08-06-79
- [9] C. D. Bruzewicz, J. Chiaverini, R. McConnell y J. M. Sage, "Trapped-ion quantum computing: Progress and challenges," Applied Physics Reviews, vol. 6, no. 2, p. 021314, 2019. doi: 10.1063/1.5088164
- [10] P. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," SIAM Journal on Computing, vol. 26, no. 5, pp. 1484–1509, 1997. doi: 10.1137/S0097539795293172
- [11] QRydDemo Consortium, "QRydDemo: A Rydberg-atom quantum computing demonstrator," University of Stuttgart and University of Kaiserslautern-Landau, 2023. Available: <https://qryddemo.itp3.uni-stuttgart.de/>

- [12] M. AbuGhanem, «Comprehensive characterization of three-qubit Grover search algorithm on IBM's 127-qubit superconducting quantum computers», arXiv:2406.16018, 2024.
- [13] R. Pérez-Castillo y M. Piattini, «Design of classical-quantum systems with UML», Computing, vol. 104, n.º 11, pp. 2375–2403, 2022. doi: 10.1007/s00607-022-01091-4.
- [14] N. Innan et al., «Feasibility and limitations of generalized Grover search-based quantum asymmetric cryptography: an implementation study on quantum hardware», Electronics, vol. 14, n.º 19, art. 3821, 2025. doi: 10.3390/electronics14193821.

Anexos

[A] Código orientador (Búsqueda clásica).

```
"""Búsqueda clásica por fuerza bruta
(código orientador)."""
import random
import time

BITS = 10 # cambiar según el caso de
estudio

def busqueda_clasica(bits):
    espacio_busqueda = 2 ** bits
    llave_objetivo = random.randint(0,
    espacio_busqueda - 1)
    inicio = time.perf_counter()
    iteraciones = 0
    for intento in
range(espacio_busqueda):
        iteraciones += 1
        if intento == llave_objetivo:
            break
    fin = time.perf_counter()
    return {"bits": bits, "iteraciones":
iteraciones, "tiempo": fin - inicio}

if __name__ == "__main__":
    resultado = busqueda_clasica(BITS)
    print(resultado)

}
```

[B] Flujo Código orientador (Grover) - pseudocódigo.

```
# 1. Generar llave objetivo aleatoria
llave = generar_llave(bits)
# 2. Construir oráculo MCPPhase sin
ancilla
oracle = construir_oraculo(bits, llave)
# 3. Calcular  $k^* = \text{floor}((\pi/4) * \sqrt{2^{**bits}})$ 
k_teorico = floor((pi/4) * sqrt(2**bits))
# 4. Submuestrear hasta
MAX_EVALUACIONES_K candidatas de k
k_candidatos =
generar_iteraciones_candidatas(k_teorico)
# 5. Evaluar cada k
for k in k_candidatos:
    circuito =
grover_operator(oracle).power(k)
    medir(circuito, shots=1024)
    if frecuencia_llave >= umbral:
        registrar_exito(k)
# 6. Registrar métricas extendidas en CSV
guardar(iteraciones, profundidad, shots,
tasa_exito, distribucion)
```

[C] Resultados obtenidos durante el primer avance.

C.1. $n = 2$:

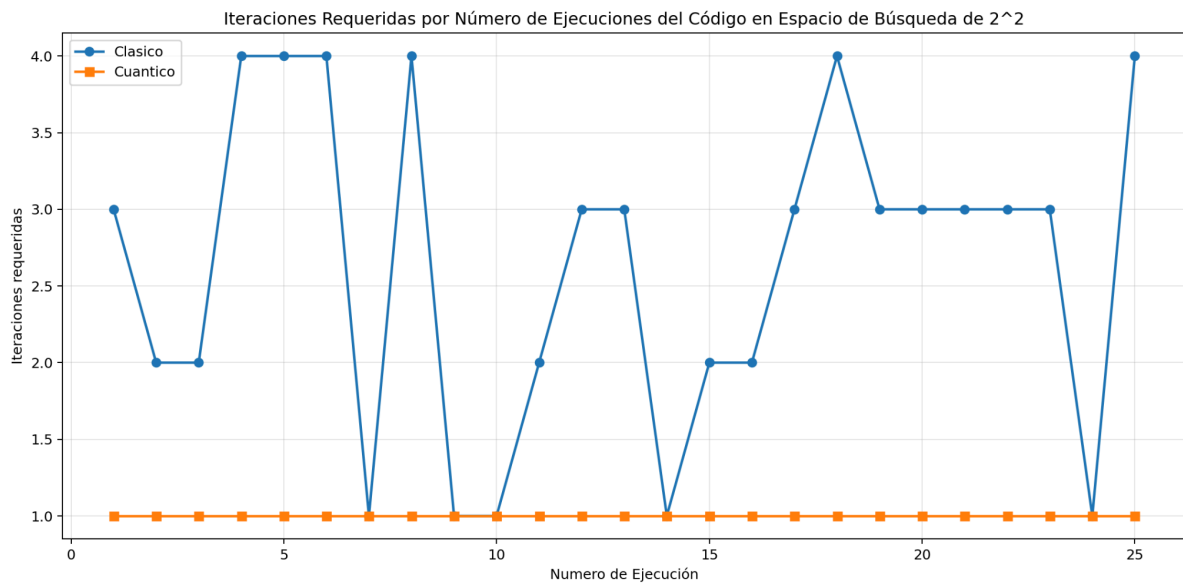


Figura C.1 - Iteraciones $n=2$

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	2.64	0.000002
Cuántico	1.00	0.375563

Tabla C.1 - Resultados espacio 2^2

C.2. n=6:

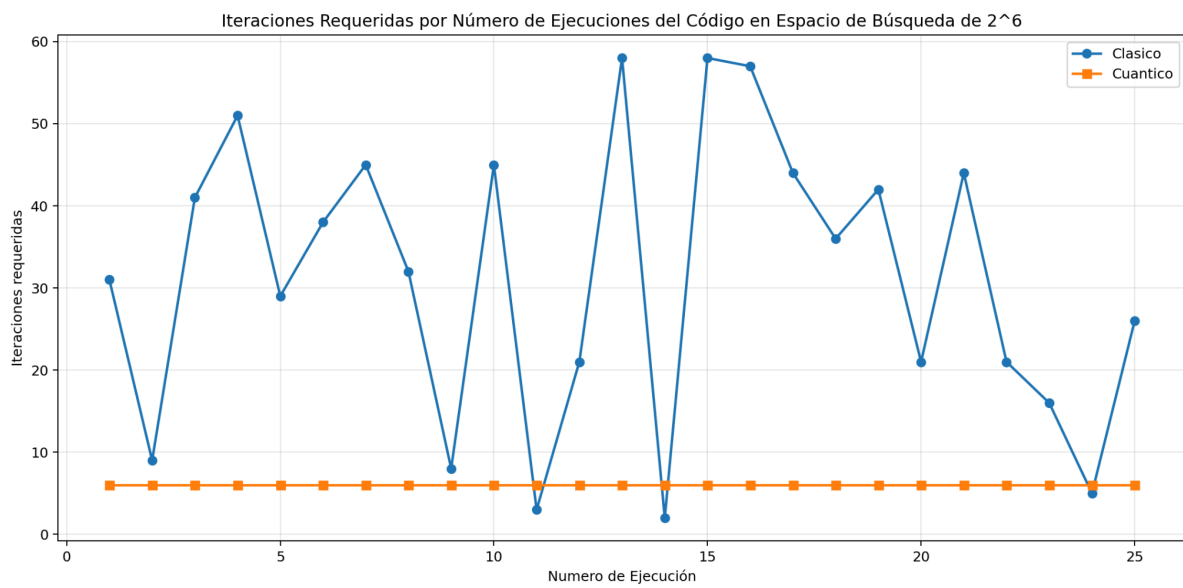


Figura C.2 - Iteraciones n=6

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	31.32	0.000003
Cuántico	6.00	2.168292

Tabla C.2 - Resultados espacio 2^6

C.3. n=8:

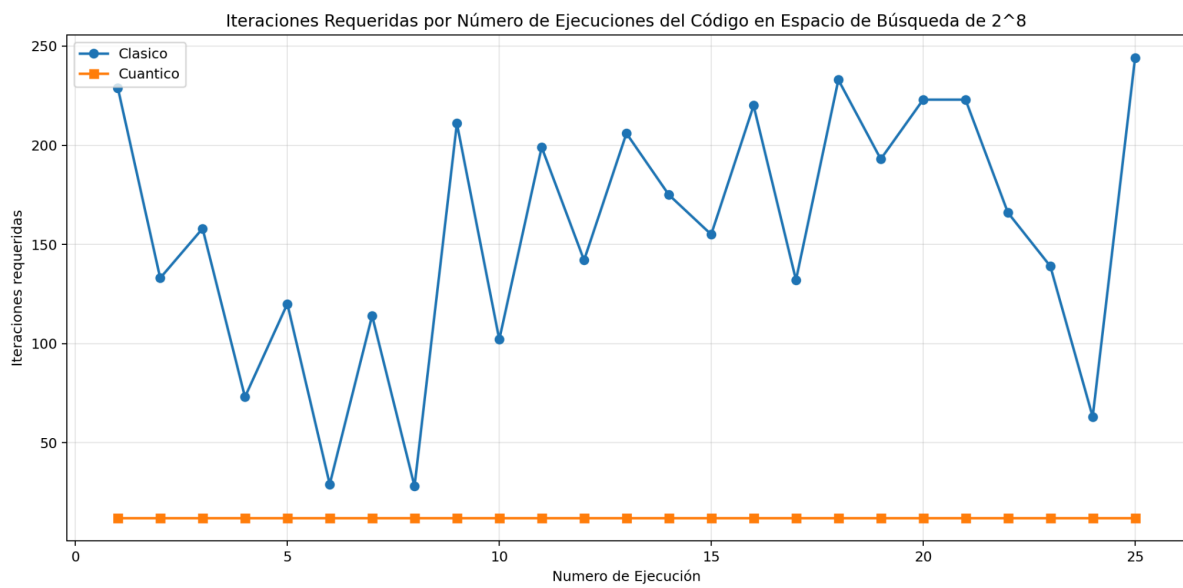


Figura C.3 - Iteraciones n=8

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	156.40	0.000008
Cuántico	12.00	4.611284

Tabla C.3 - Resultados espacio 2^8

C.4. n=10:

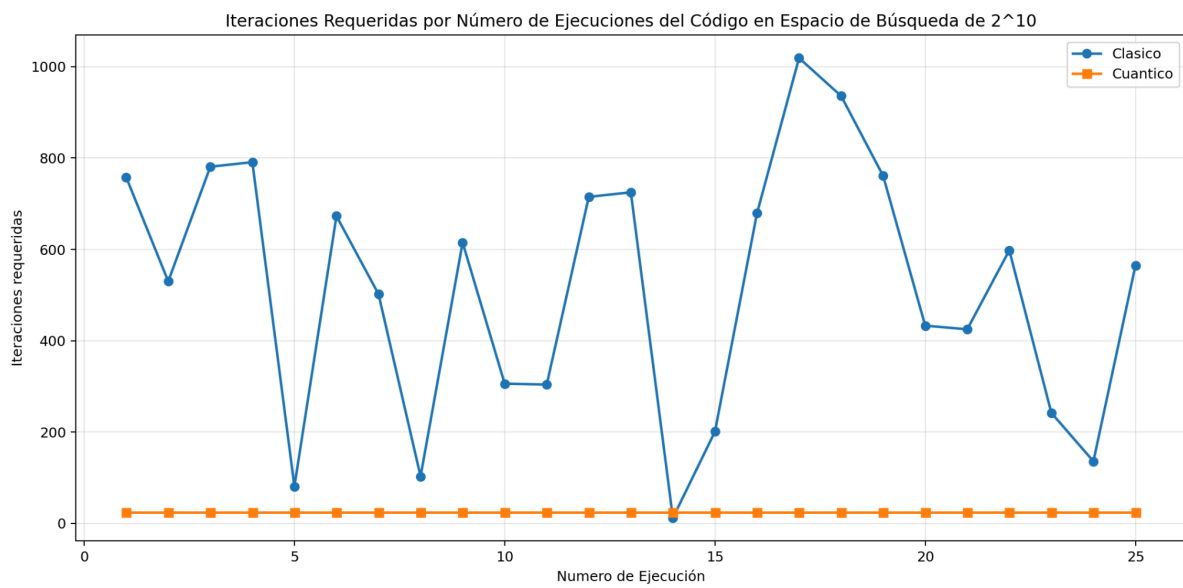


Figura C.4 - Iteraciones por ejecución

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	515.64	0.000037 s
Cuántico	24.00	5.214571 s

Tabla C.4 - Resultados espacio 2^{10}

C.5. n=12:

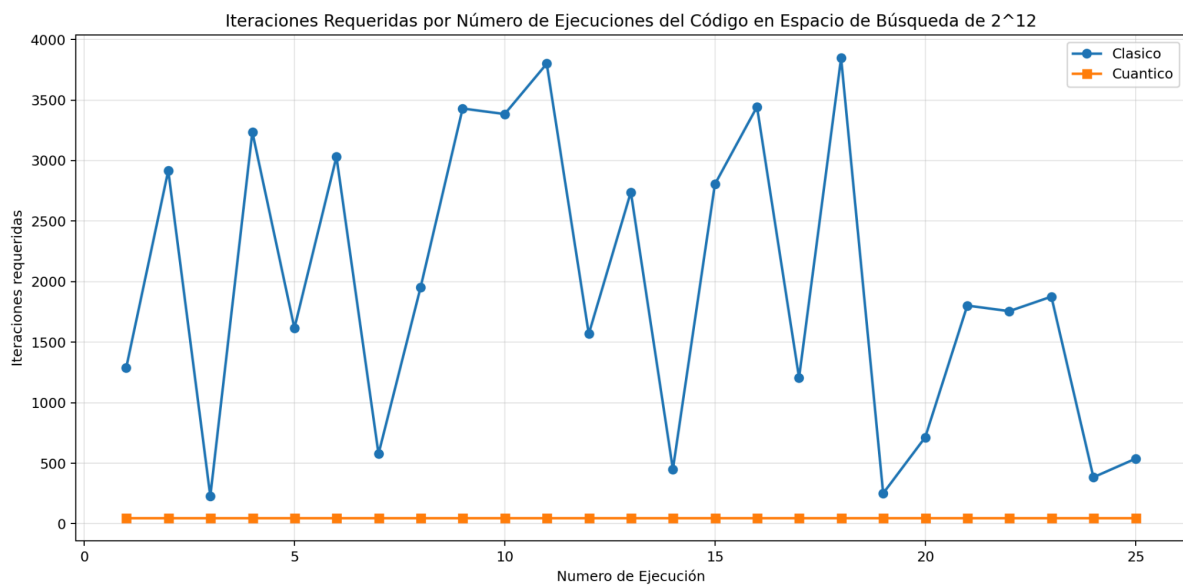


Figura C.5 - Iteraciones n=12

Modelo	Iteraciones promedio	Tiempo promedio
Clásico	1953.52	0.000138
Cuántico	48.00	7.112998

Tabla C.5 - Resultados espacio 2^{12}

[D] Resultados obtenidos durante el avance 2.

D.1.

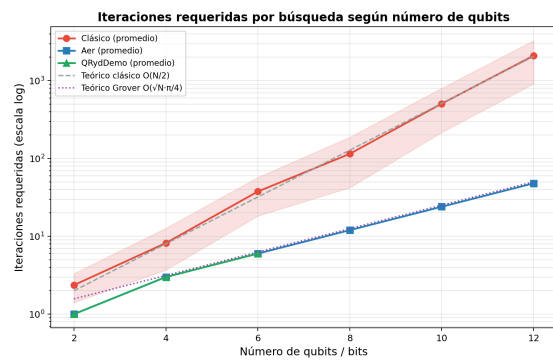


Figura D.1.1 - Iteraciones por búsqueda

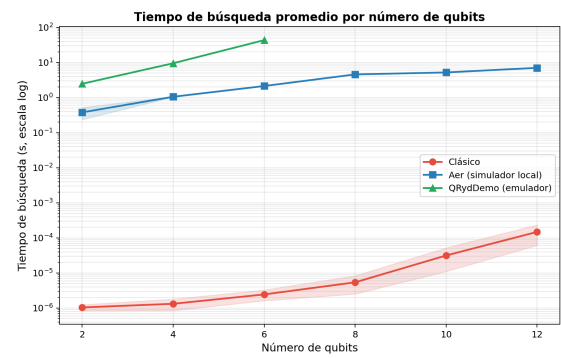


Figura D.1.2. - Tiempo de búsqueda promedio

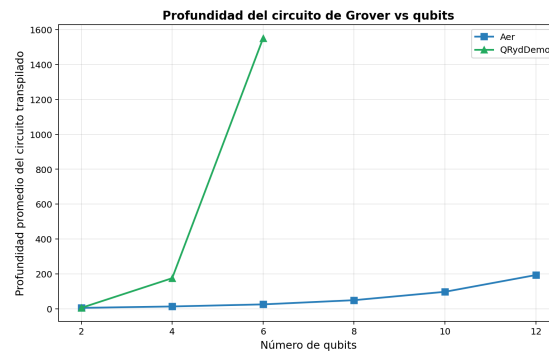


Figura D.1.3. - Profundidad del circuito

D.2.

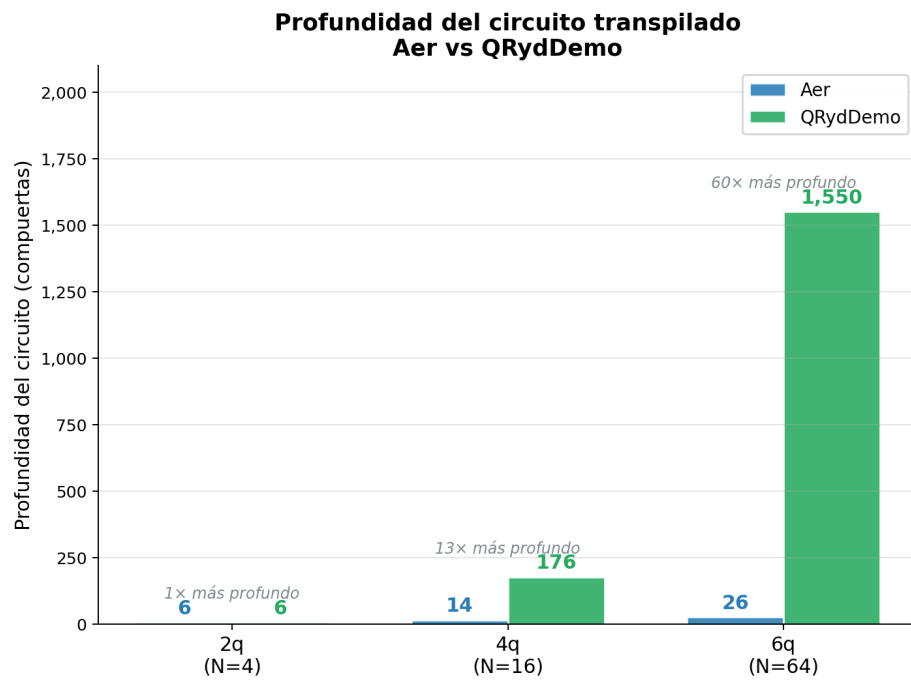


Figura D.2 - Profundidad del Circuito

D.3.

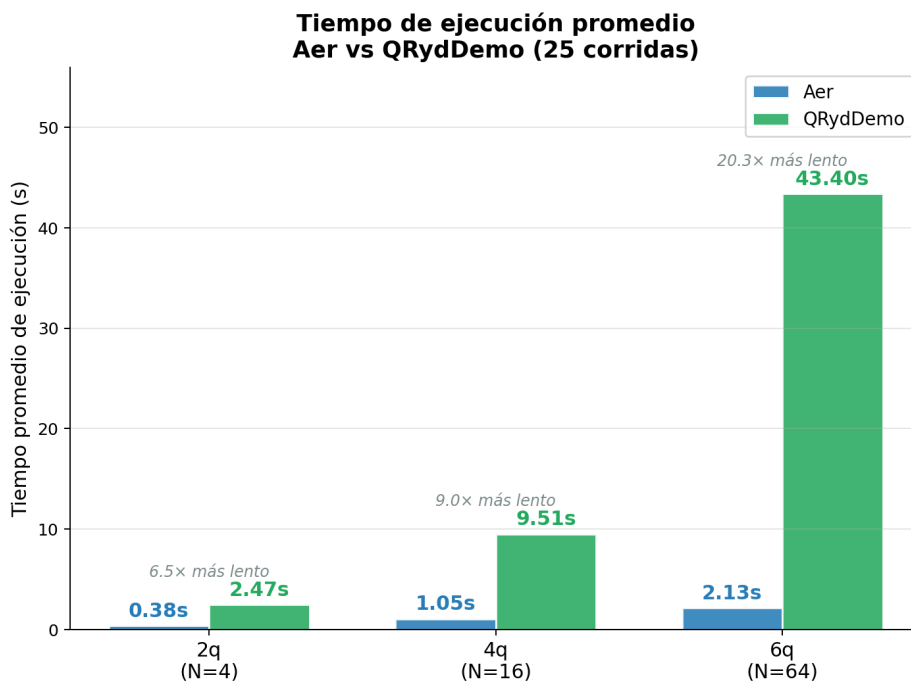


Figura D.3 - Tiempo de Ejecución Promedio

D.4.

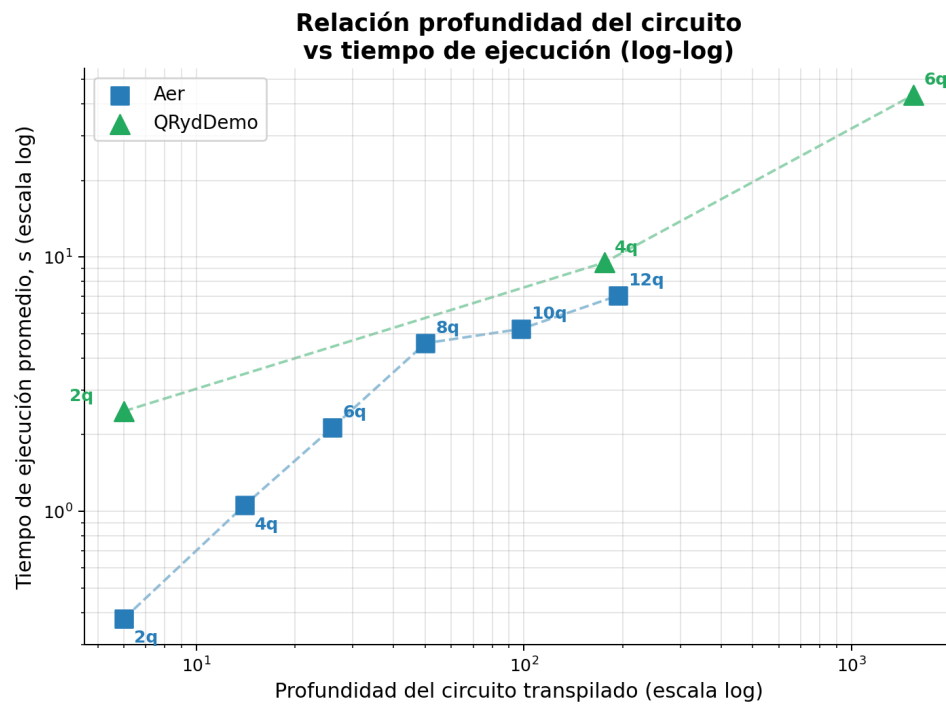


Figura D.4 - Profundidad del Circuito vs Tiempo de Ejecución

D.5.

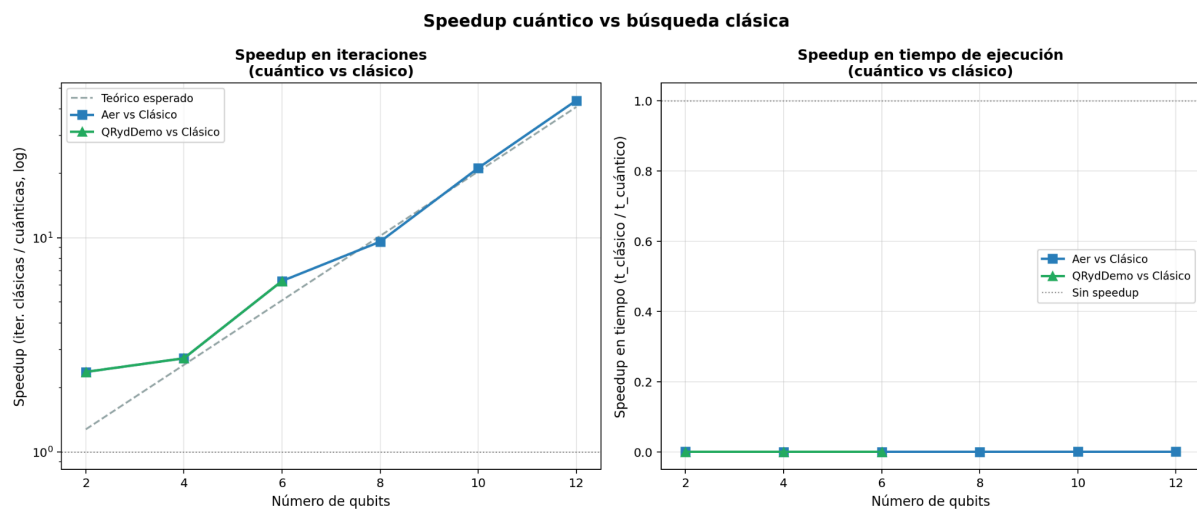


Figura D.5 - Speedup cuántico vs búsqueda clásica

D.6.

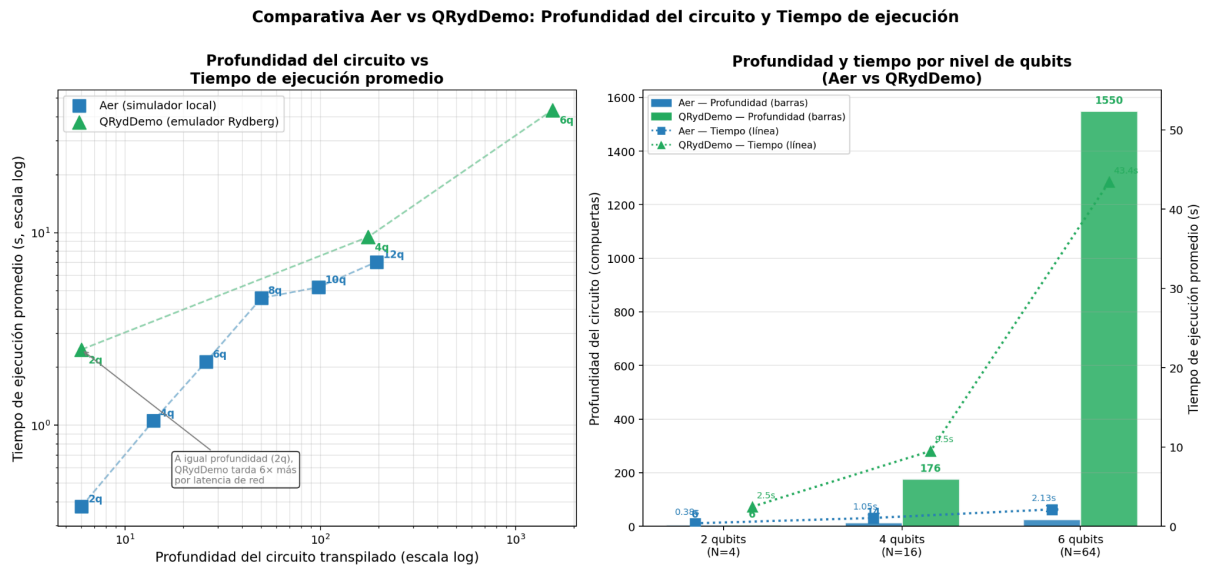


Figura D.6 - Profundidad del circuito y tiempo de ejecución

D.7.

Tiempo promedio de ejecución por nivel de qubits — Clásico vs Aer vs QRydDemo

Qubits	N	Clásico (s)	Aer (s)	QRydDemo (s)	Aer / Clásico	QRyd / Clásico
2q	4	0.000001 s	0.3785 s	2.4743 s	378,476x	2,474,319x
4q	16	0.000001 s	1.0547 s	9.5056 s	1,054,667x	9,505,569x
6q	64	0.000002 s	2.1346 s	43.4034 s	1,067,280x	21,701,715x
8q	256	0.000005 s	4.5785 s	—	915,707x	—
10q	1,024	0.000031 s	5.2093 s	—	168,042x	—
12q	4,096	0.000148 s	7.0513 s	—	47,644x	—

Tabla D.1 - Tiempo de ejecución promedio por nivel de qubits

D.8. Tablas resultados por tipo de ejecución:

Bits	N	Corridas	Iter. Promedio	Mediana	Desv. Est.	Tiempo Prom. (s)	Iter. Esperadas (N/2)	Razón vs Esperado
2	4	25	2.36	2	0.95	1×10^{-6}	2	1.18
4	16	25	8.20	7	4.51	1×10^{-6}	8	1.03
6	64	25	37.64	43	19.38	2×10^{-6}	32	1.18
8	256	25	114.88	97	72.65	5×10^{-6}	128	0.90
10	1024	25	507.32	544	291.14	3.1×10^{-5}	512	0.99
12	4096	25	2098.04	2028	1185.11	1.48×10^{-4}	2048	1.02

Tabla D.2 - Resultados de búsqueda clásica

Bits	N	Iter. Prom.	Iter. Teórica	Razón	Éxito (%)	Profundidad	Evaluaciones	Speedup
2	4	1	1	1.00	100.00	6	2	2.36
4	16	3	3	1.00	97.34	14	6	2.73
6	64	6	6	1.00	99.78	26	12	6.27
8	256	12	12	1.00	100.00	50	24	9.57
10	1024	24	25	0.96	99.82	98	24	21.14
12	4096	48	50	0.96	99.76	194	24	43.71

Tabla D.3 - Resultados del algoritmo de Grover (Aer)

Bits	N	Iter. Prom.	Éxito (%)	Tiempo Prom. (s)	Profundidad	Compuertas	Entropía	Speedup
2	4	1	100.00	2.47	6	10	0.000	2.36
4	16	3	95.73	9.51	176	223.4	0.416	2.73
6	64	6	99.86	43.40	1550	2064	0.017	6.27

Tabla D.4 - Resultados en Qryd (hasta 6 qubits)

[E] Explicación completa oráculo

El oráculo cuántico: diseño, implementación y justificación

El oráculo constituye el componente central y más crítico del algoritmo de Grover. Es la única parte del circuito que codifica el conocimiento del problema y, por tanto, determina si la ventaja cuántica es alcanzable en la práctica. Su función matemática es aplicar un cambio de fase global de π al estado objetivo $|\omega\rangle$ y dejar inalterados todos los demás estados:

$$O|\omega\rangle = -|\omega\rangle \quad ; \quad O|x\rangle = |x\rangle \quad \text{si } x \neq \omega$$

Esta operación es reversible (unitaria) y no mide ni colapsa ningún estado, lo cual es un requisito fundamental de los circuitos cuánticos.

Implementación concreta en el código

La función `construir_oraculo(bits, llave)` del módulo `QRydDemo` implementa el oráculo de fase sin qubits ancilla mediante tres etapas reversibles:

Paso 1 - Normalización al estado $|11\dots 1\rangle$: Se aplica una compuerta X (NOT) sobre cada qubit cuyo bit correspondiente en la llave objetivo es '0'. Esto transforma el estado objetivo en $|11\dots 1\rangle$, independientemente de cuál sea la llave. Si la llave es, por ejemplo, '1011' en un sistema de 4 qubits, se aplica X solo al qubit cuyo bit es '0' (posición 2), convirtiendo '1011' en '1111'.

Paso 2 - Aplicación de $\text{MCPhase}(\pi)$: Se aplica una compuerta $\text{MCPhase}(\pi)$ con $n-1$ controles y un qubit objetivo. Esta compuerta multi-controlada introduce la fase -1 única y exclusivamente cuando los n qubits están todos en el estado $|1\rangle$ simultáneamente, es decir, cuando el sistema se encuentra en el estado $|11\dots 1\rangle$ transformado. El efecto neto es: $|11\dots 1\rangle \rightarrow -|11\dots 1\rangle$.

Paso 3 - Restauración de la base: Se invierten las compuertas X del Paso 1 (misma secuencia de qubits). Esto restaura el estado objetivo a su representación original en la base computacional, dejando el circuito en condiciones idénticas a las iniciales para todos los estados salvo el objetivo, que acumuló la fase -1 .

El pseudocódigo equivalente es:

```
def construir_oraculo(bits: int, llave: str) -> QuantumCircuit:
    oracle = QuantumCircuit(bits)
    # Paso 1: X en qubits con bit '0' en la llave
    for qubit in range(bits):
        if llave[bits - 1 - qubit] == '0':
            oracle.x(qubit)
    # Paso 2: MCPhase(pi) con n-1 controles -> fase -1 en |11...1>
    oracle.mcp(pi, list(range(bits - 1)), bits - 1)
    # Paso 3: Invertir X para restaurar la base computacional
    for qubit in range(bits):
        if llave[bits - 1 - qubit] == '0':
            oracle.x(qubit)
    return oracle
```

Por qué se eligió este oráculo

La elección del oráculo MCPPhase sin qubits ancilla responde a cuatro criterios técnicos fundamentales:

- Reversibilidad garantizada: Las compuertas X y MCPPhase(π) son unitarias e inversas unas de otras, por lo que el circuito cumple el requisito de reversibilidad.
- Ausencia de qubits auxiliares: El oráculo utiliza exactamente n qubits para un espacio de búsqueda de 2^n . Esto minimiza el overhead de recursos y permite escalar el experimento hasta el límite del simulador sin penalidad adicional.
- Profundidad $O(n)$ por llamada: El total de operaciones por ejecución del oráculo es $2n$ compuertas X más una MCPPhase, que se descompone en $O(n)$ compuertas Toffoli anidadas durante la transpilación. Esto resulta en una profundidad $O(n)$ por oráculo, la mínima posible para una implementación directa sin preprocesamiento.
- Compatibilidad con QRYdDemo y Qiskit: La MCPPhase(π) está disponible directamente en Qiskit como mcp y es descomponible por el transpilador en el conjunto nativo de compuertas del backend QRYdDemo ("qryd_emulator\$square"), lo que hace el oráculo portable entre simuladores.

Análisis de escalamiento del oráculo

Qubits (n)	Comp. X (paso 1+3)	MCPPhase descomp.	Profundidad	Qubits aux.
2	2	$O(1)$	$O(1)$	0
4	Hasta 4	$O(n)$	$O(n)$	0
6	Hasta 6	$O(n)$	$O(n)$	0
10	Hasta 10	$O(n)$	$O(n)$	0
n	Hasta n	$O(n)$ Toffoli	$O(n)$	0

Tabla E.1 - Escalamiento de recursos del oráculo con el número de qubits.

Un aspecto crítico a considerar es que, aunque el número de consultas al oráculo escala como $O(\sqrt{N})$, la profundidad total del circuito escala como $O(n \cdot 2^{n/2})$. Esto significa que para $n = 10$, la profundidad total es proporcional a $10 \times \sqrt{1024} \approx 320$ capas de compuerta, lo que excede los límites de coherencia de la mayoría de dispositivos NISQ para $n \geq 5$ qubits.

[F] Resultados avance final

Link repositorio proyecto, creado durante este avance con archivos específicos por ejecución, tablas, .csv e imágenes adicionales generadas para los 3 avances del proyecto:

<https://github.com/Antix199/Q-Search>

Tabla comparativa: Clásico vs Aer vs QRYdDemo vs IQM (hardware real)

Qubits	N	Clás. iter. (prom)	Aer iter. (prom)	QRyd iter. (prom)	IQM iter. (prom)	Grover teórico	Speedup iter. Aer/Clas	Speedup iter. QRyd/Clas	Speedup iter. IQM/Clas	Tasa éxito Aer (%)	Tasa éxito QRyd (%)	Tasa éxito IQM (%)	Acierto top-1 Aer (%)	Acierto top-1 QRyd (%)	Acierto top-1 IQM (%)
2	4	2.5	1.0	1.0	1.0	1	2.48x	2.48x	2.48x	100.0	100.0	97.3	100.0	100.0	100.0
4	16	8.7	3.0	3.0	1.8	3	2.89x	2.89x	4.93x	97.3	95.7	26.4	100.0	100.0	100.0
6	64	33.6	6.0	6.0	—	6	5.60x	5.60x	—	99.8	99.9	—	100.0	100.0	—
8	256	107.5	12.0	—	—	12	8.96x	—	—	100.0	—	—	100.0	—	—
10	1,024	498.8	24.0	—	—	25	20.78x	—	—	99.8	—	—	100.0	—	—
12	4,096	2073.1	48.0	—	—	50	43.19x	—	—	99.8	—	—	100.0	—	—

Tabla F.1 - Comparativa ejecución entorno clásico vs Aer vs QRYdDemo vs IQM.

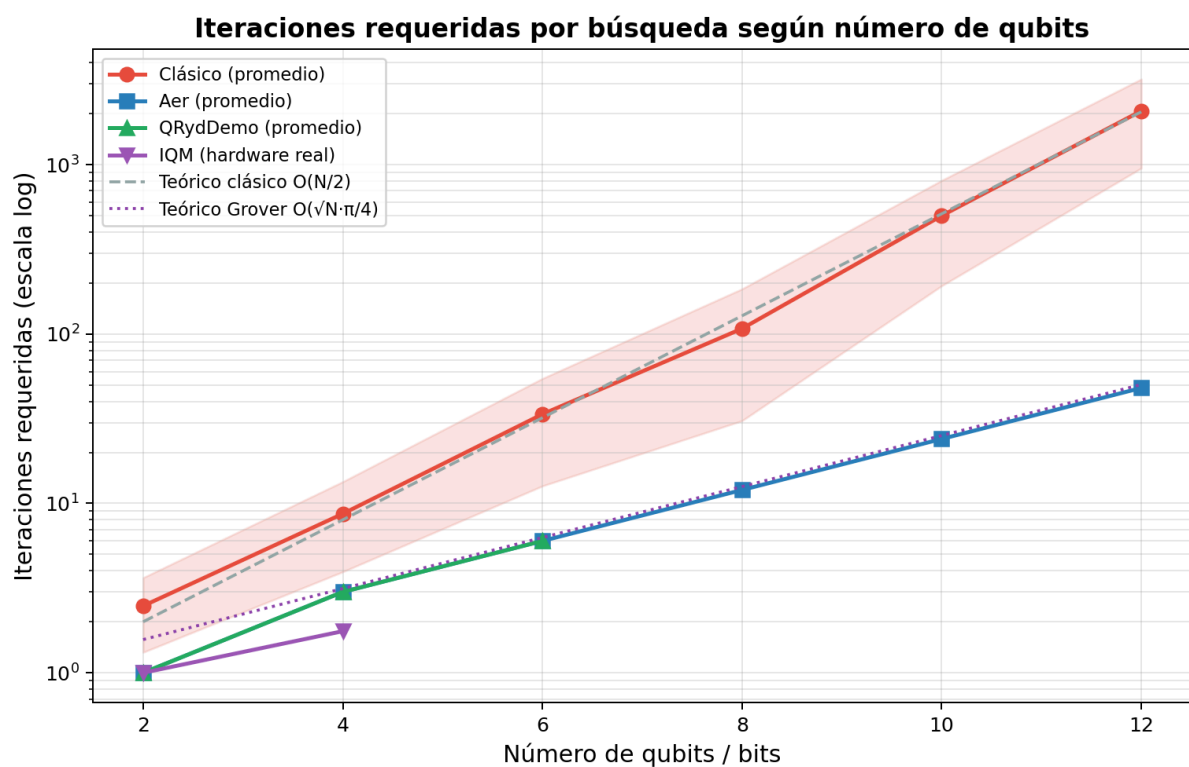


Figura F.1 - Iteraciones requeridas por búsqueda.

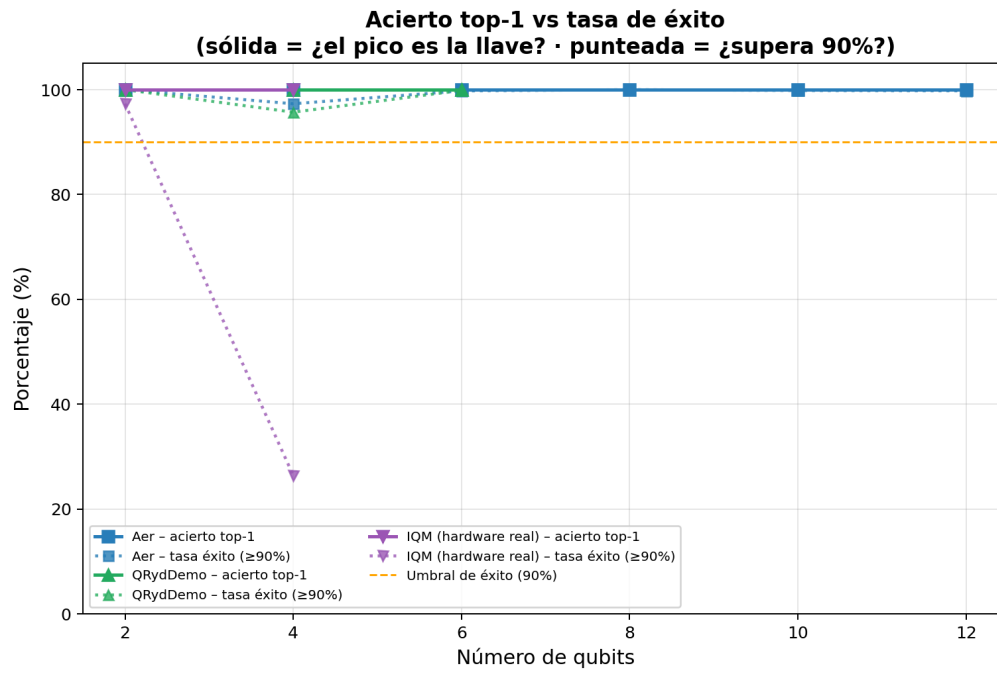


Figura F.2. - Acierto top-1 vs tasa de éxito.

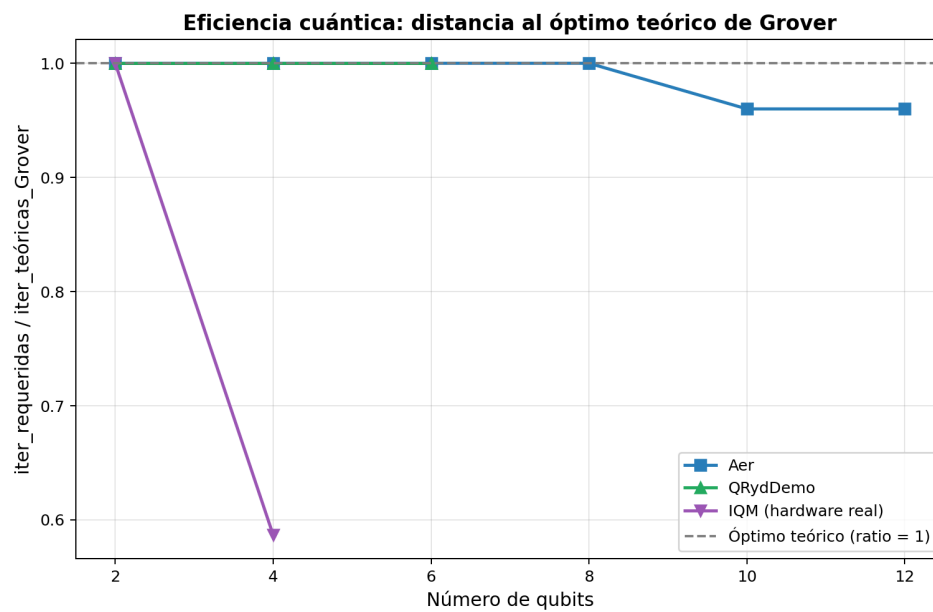


Figura F.3. - Eficiencia cuántica.

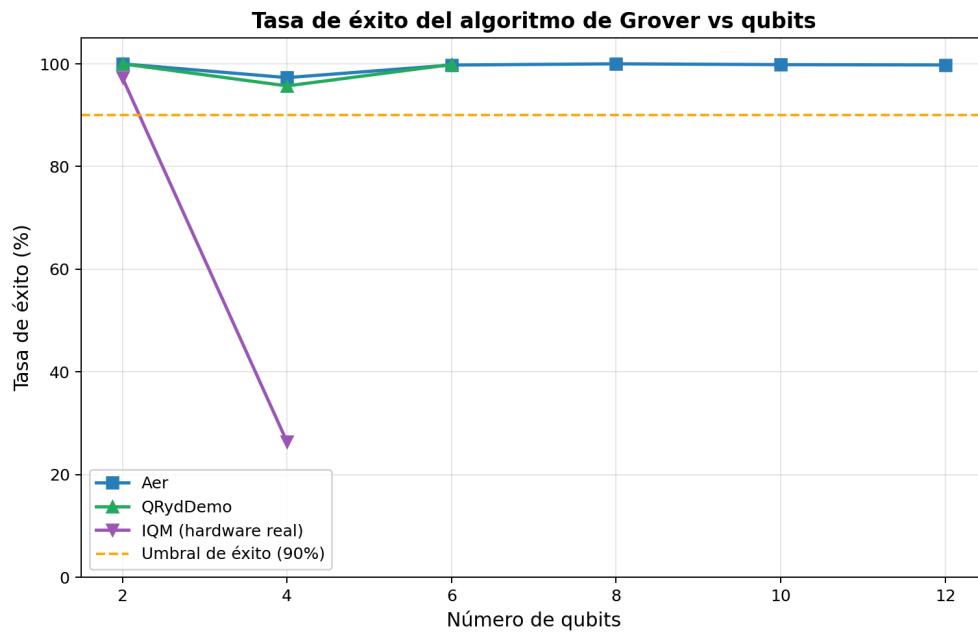


Figura F.4. - Tasa de éxito algoritmo.

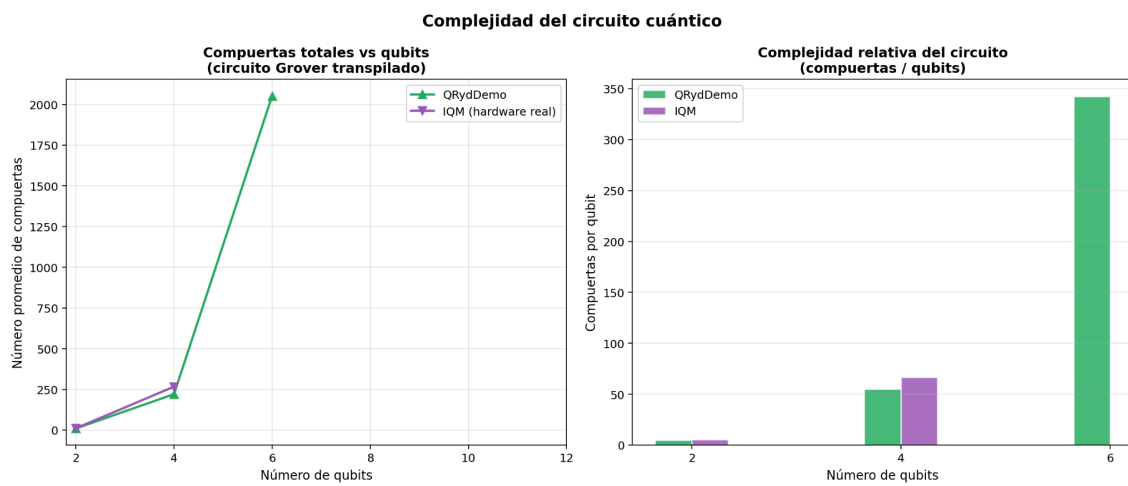


Figura F.5. - Complejidad circuito cuántico.

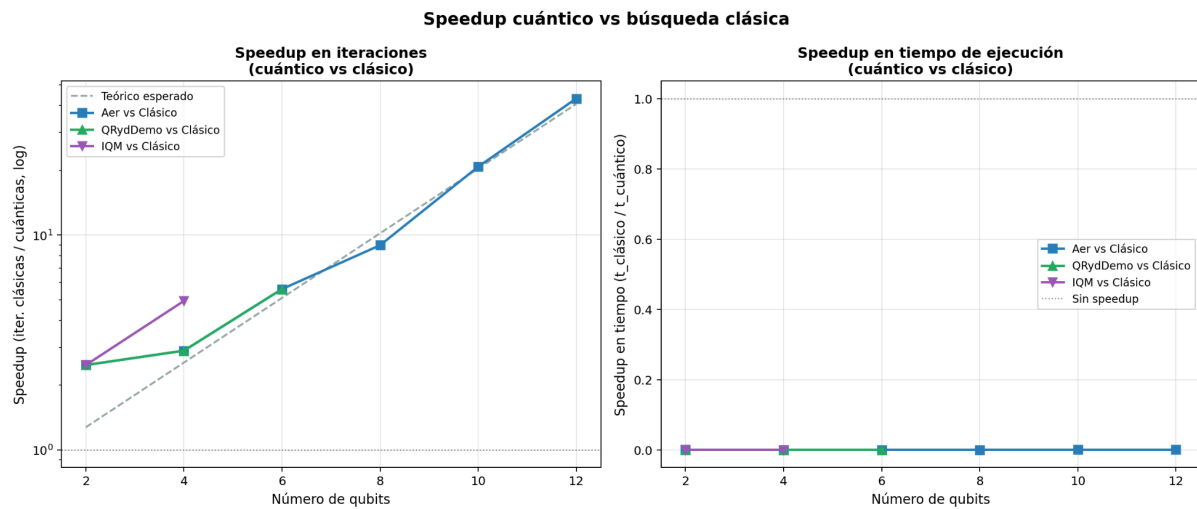


Figura F.6. - Speedup cuántico vs clásico.

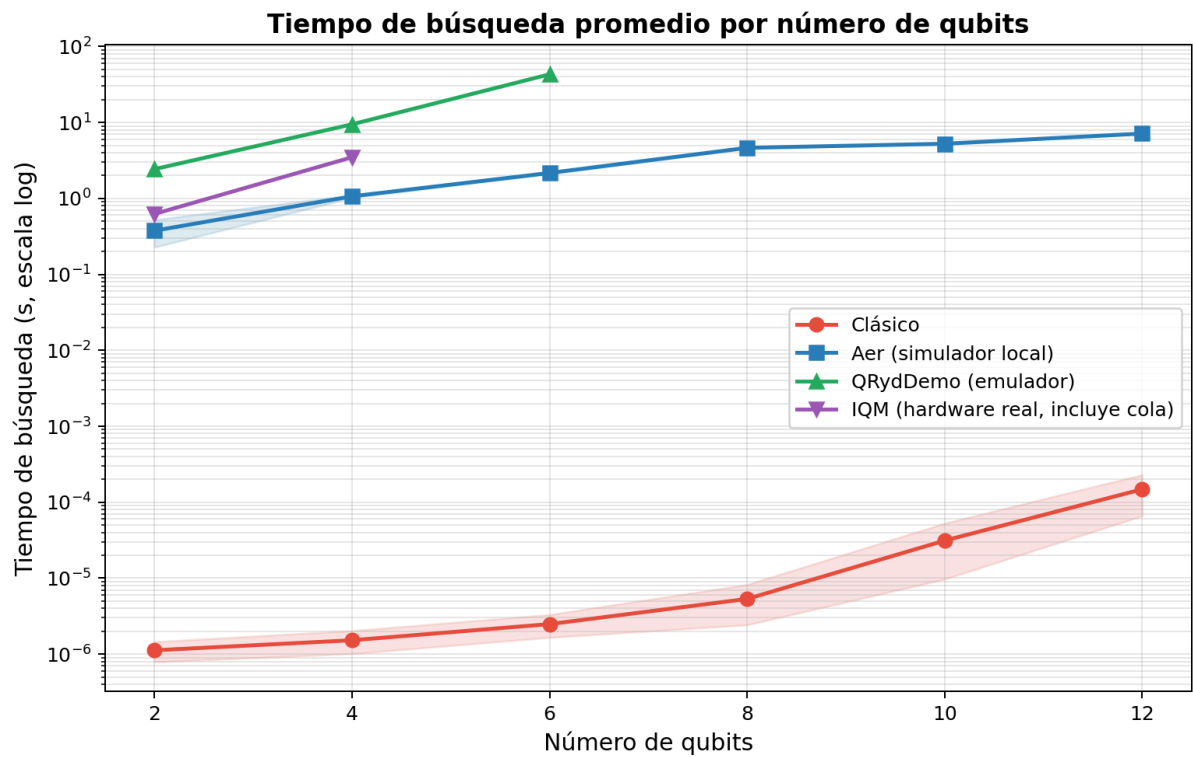


Figura F.7.- Tiempo de búsqueda promedio por número de qubits.

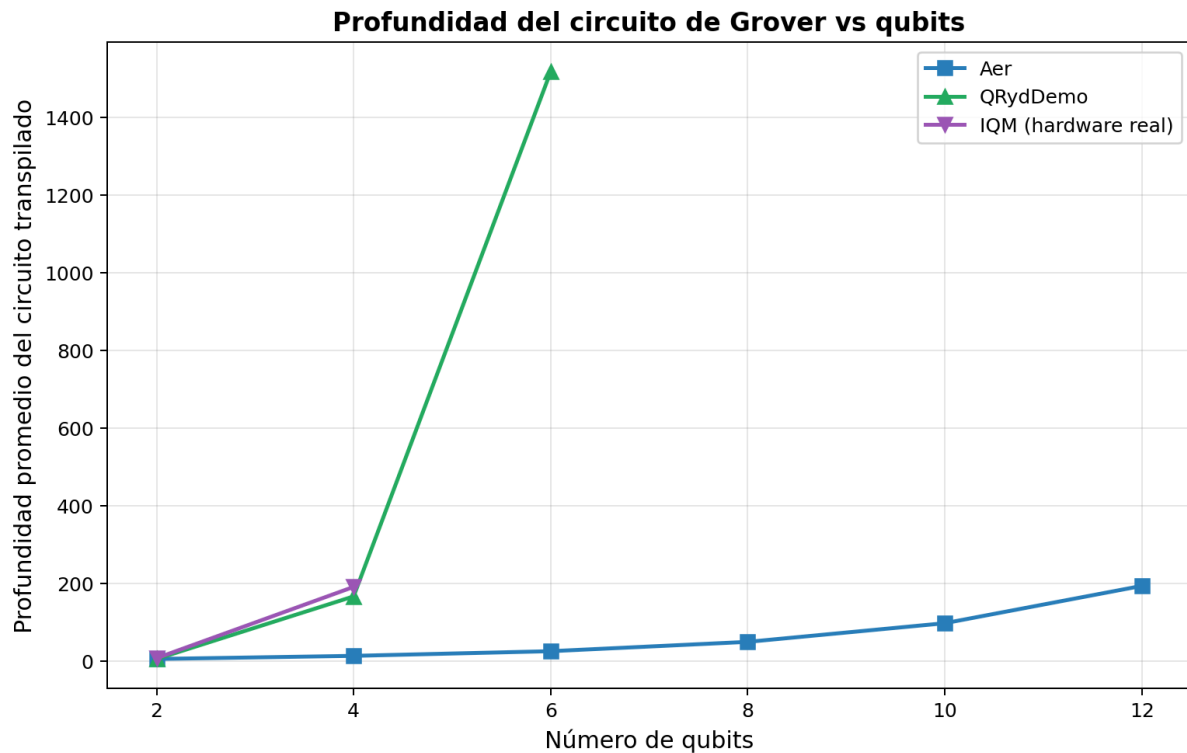


Figura F.8. - Profundidad del circuito.

[G] Perfil del hardware real (IQM Emerald)

Parámetro	Valor aproximado
Plataforma	IQM Resonance (plan freemium)
Backend / dispositivo	IQM Emerald (superconductor)
Compuertas nativas	prx, cz
Error por CZ	~0,6 %
Tiempo de coherencia T_2	~36 μ s
Niveles ejecutados	n = 2 (1 job) y n = 4 (2 jobs: 100+50 circuitos)

Tabla G.1 — Parámetros del entorno de ejecución en hardware real.

[H] Uso de IA generativa

Conforme a la pauta, se declara el uso de IA generativa durante el desarrollo. Herramienta: Claude (Anthropic), a través de Claude Code (modelo de la familia Claude). Actividades realizadas con asistencia de la herramienta:

1. Depuración del módulo de ejecución en hardware real `quantum_search_iqm.py` (integración con `iqm-client[qiskit]`, transpilación a ISA y troceado de jobs);
2. Apoyo en el análisis e interpretación de los resultados de 4q en IQM Emerald (distinción entre acierto top-1 y tasa de éxito, y su explicación por decoherencia).
3. Apoyo en la revisión ortográfica y gramatical del presente informe.